

Técnicas de programación

Índice de contenidos

- Técnicas de programación
 - Índice de contenidos
 - Movimiento de datos
 - Errores frecuentes
 - Aritmética
 - Restar del acumulador
 - Realizar la operación $w - 1$
 - Las banderas
 - Las banderas en la suma
 - Las banderas en la resta
 - Las banderas en la rotación
 - Operaciones de comparación
 - Igualdad
 - Mayor que y menor que
 - Subrutinas
 - La instrucción CALL
 - Consulta a tablas
 - Conversión a ASCII
 - Ramificación múltiple
 - Solución 1
 - Solución 2
 - Temporización
 - Instrucciones y puertos
 - Escritura en los puertos
 - Cuestiones a tener en cuenta
 - Lectura de los bits del puerto
 - Pulsadores y anti-rebotes
 - Hardware para evitar rebotes
 - Software para evitar rebotes
 - Otras operaciones en los puertos

Movimiento de datos

El juego de instrucciones reducido, y su tamaño de 14 bits, hacen que el PIC16F84A tenga una serie de restricciones. Por un lado no se pueden especificar dos registros dentro de una instrucción. Cada registro necesita 7 bits para especificar la dirección, pero también hay que especificar el código de la instrucción y qué hacer con ella. La solución es realizar todo a través del registro de trabajo o **w** que no necesita dirección y está situado dentro de la CPU del microcontrolador. Una transferencia de un registro a otro necesitaría dos instrucciones. Supongamos que tenemos que transferir un dato al puerto B:

<u>MOVE</u>	DATO, W ; copia el contenido del registro DATO en W
<u>MOVWF</u>	PORTB ; copia el contenido de W en el Puerto B

Nota: En todos los ejemplos consideramos que **w** = 0 y que **f** = 1, esto significa que [MOVWF DATO, W](#) es lo mismo que [MOVWF DATO, 0](#)

La primera instrucción tiene la forma [MOVWF f,d](#) que copia el registro **f** en el destino especificado por **d** (**w** en este caso). La segunda instrucción simplemente mueve cualquier dato contenido en **w** en el registro **f**, que en este caso es el puerto B.

El registro **DATO** permanece invariable en la primera instrucción y **w** permanece invariable en la segunda, de manera que estas instrucciones se parecen más a una copia que a un movimiento de datos.

Las instrucciones con literales no tienen espacio para contener la dirección de un registro, por eso debemos utilizar el registro de trabajo **w** para cargar un registro con un literal y también se necesitan dos instrucciones.

```
MOVLW    0xAA    ; coloca el valor 10101010 en W
MOVWF    DATO    ; copia W en el registro DATO
```

Esto mismo se aplica cuando se usan operaciones booleanas, de suma y de resta entre literales y registros. Todas necesitan dos instrucciones:

```
MOVLW    k        ; copia el literal en W
SUBWF    f,d      ; copia el resultado de restar W de f en d
```

Supongamos que queremos poner a cero el nibble inferior

```
MOVLW    0xF0      ; ponemos una mascara ('11110000')
ANDWF    DATO, f    ; el resultado de DATO AND 0xF0 se coloca en DATO
```

Las instrucciones de un solo operando son fáciles de entender:

- [CLRF f](#), **Pone todos los bits del registro f a cero**
- [CLRWF](#), **Pone todos los bits de W a cero**
- [BCF f,b](#), **Pone a cero el bit b del registro f**
- [BSF f,b](#), **Pone a uno el bit b del registro f**

Errores frecuentes

Es fácil cometer pequeños errores que nos harán gastar gran cantidad de tiempo. Aquí hay algunas causas de problemas frecuentes.

Muchas instrucciones de un programa son del tipo **MOV** y están relacionadas con **w**. Es muy fácil confundir "cargar un registro en **w**" con "cargar **f** con **w**".

- [MOVWF f](#), **w se mueve al registro f (El resultado se guarda en f)**
- [MOVWF f, w](#), **El registro f se mueve a w, guardando el resultado en w**
- [MOVWF f, f](#), **El registro f se mueve sobre sí mismo**

En este ultimo caso, el registro no varía, pero las banderas del registro [STATUS](#) si.

[MOVWF](#) es la única instrucción **w-f** que no tiene bit de destino, ya que el destino siempre será **f**.

Las instrucciones **w-f** son:

- [ADDWF f,d](#), **Suma el valor de w al registro f guardándolo en w o f**
- [ANDWF f,d](#), **AND del valor de w con el registro f guardándolo en w o f**
- [IORWF f,d](#), **OR del valor de w con el registro f guardándolo en w o f**
- [MOVWF f](#), **Mueve el valor de w al registro f guardándolo en w o f**
- [SUBWF f,d](#), **Resta al valor del registro f el valor de w guardándolo en w o f**
- [SWAPF f,d](#), **Intercambia los nibbles del registro f guardándolo en w o f**
- [XORWF f,d](#), **OR Exclusiva del valor de w con el registro f guardándolo en w o f**

En todos estos casos, **w** o **f** cambiarán, según el destino.

Otras instrucciones cuyo destino cambia son:

- [INC f,d](#), **Incrementa el valor del registro f guardándolo en w o f**
- [DEC f,d](#), **Decrementa el valor del registro f guardándolo en w o f**
- [COMP f,d](#), **Complementa el valor del registro f guardándolo en w o f**
- [SWAP f,d](#), **Intercambia el valor del registro f guardándolo en w o f**
- [RLF f,d](#), **Rota a la izquierda el valor del registro f guardándolo en w o f**
- [RRF f,d](#), **Rota a la derecha el valor del registro f guardándolo en w o f**

Otro error común es poner [GOTO](#) cuando deberíamos poner [CALL](#) y viceversa. Esto provocaría que el programa se quede colgado o se comporte de manera extraña. Relacionados con el mismo tipo de instrucciones, otro error común es olvidar poner al final de las subrutinas la instrucción de retorno [RETURN](#), [RETLW](#) o [RETFIE](#).

Un problema que puede darse con las rutinas del tipo **addwf PCL,1** es que se encuentren situadas mas alla de la dirección de memoria de programa 255. Para solucionarlo basta con tomar por costumbre colocar las rutinas al principio del programa y que el tipo de rutina citado no supere la posición de memoria 255. En el ejemplo siguiente la rutina **CODIGO_7S** dará problemas si no se sitúa al principio:

```
;*****
    ORG 0x00                ;Vector de Reset
    goto    INICIO
    org     0x05            ;Salva el vector de interrupción
;*****
; SUBROUTINAS
;*****
CODIGO_7S      ; Devuelve el código 7 segmentos
    addwf    PCL,1
    retlw    CERO
    retlw    UNO
    retlw    DOS
    retlw    TRES
    retlw    CUATRO
    retlw    CINCO
    retlw    SEIS
    retlw    SIETE
    retlw    OCHO
    retlw    NUEVE
; .....
; .....
;*****
; Comienzo del programa
INICIO
; .....
; .....
```

Debemos tener cuidado cuando usemos los mismos registros en dos rutinas distintas, especialmente si una de ellas llama a la otra. Por ejemplo, si utilizamos **TEMP** en un bucle de temporización y después se vuelve a utilizar **TEMP** en una subrutina que llama a la subrutina de temporización, debemos tener en cuenta que la subrutina de temporización cambia **TEMP**.

Las instrucciones [RLF](#) y [RRF](#) rotan a través de **carry** del registro [STATUS](#), lo que quiere decir que el **carry** debe de ser actualizado antes de llamar a la instrucción, ya que los bits de mayor o menor peso pasarán al bit mayor o menor. Del mismo modo el bit mayor o menor será situado en el acarreo.

Aritmética

Dentro de los microcontroladores PIC se cuenta con instrucciones aritméticas tales como:

- Para efectuar operaciones de suma:
 - [ADDWF f,d](#), **Suma el valor de w al registro f guardándolo en w o f**
 - [ADDLW k](#), **Suma el valor de w al literal k guardándolo en w**
- Para efectuar operaciones de resta:
 - [SUBWF f,d](#), **Resta al valor del registro f el valor de w (f-w) guardándolo en w o f**
 - [SUBLW k](#), **Resta al valor del literal k el valor de w (k-w) guardándolo en w**
- Para realizar multiplicaciones por 2.
 - [RLF f,d](#), **Rota a la izquierda el valor del registro f guardándolo en w o f**
- Para realizar divisiones entre 2.
 - [RRF f,d](#), **Rota a la derecha el valor del registro f guardándolo en w o f**

Hasta este punto podríamos ver el conjunto de instrucciones un poco limitado. Sin embargo, utilizando las técnicas apropiadas de programación podemos obtener operaciones más complejas.

Restar del acumulador

Visto lo anterior, para restar un valor al acumulador se utiliza **ADDLW** y se le suma el complemento a 2 del valor a restar.

Realizar la operación $w - 1$

Para restar 1 al acumulador se utiliza **ADDLW 0xFF**, en lugar de **SUBLW 0x1** porque esta instrucción no resta el literal a **w**, sino al revés, al literal le resta **w**. Por lo tanto para restar un literal de **w** debemos sumar el complemento a 2 del literal con w, en nuestro caso el literal es 1 (0000 0001 b) y el complemento a 2 de 1 es FF h:

```
0000 0001
1111 1110
   +1
-----
1111 1111 (FF h.)
```

Las banderas

Las banderas se utilizan para dar información adicional cuando se realizan operaciones lógicas y aritméticas dentro del microcontrolador. Así, podremos tomar decisiones según el valor de cada una de las banderas. Existen diferentes tipos de banderas en un microcontrolador; entre ellas tenemos:

Las banderas en la suma

Los registros básicos del microcontrolador PIC16F84A tienen una longitud de 8 bits expresados en forma binaria, lo cual quiere decir que el número máximo expresado en forma decimal será el 255. En la suma existen tres tipos de banderas que pueden proporcionarnos mayor información del resultado. Estas banderas son denominadas **CARRY (C)**, **Acarreo de Dígito (DC)** y el **Estado Cero (Z)**. Todas estas banderas son activadas según sea el caso.

Por ejemplo, en la suma, la bandera **CARRY** se coloca en "1" cuando el resultado supera el número 255 y permanecerá en "0" indicando que no se presentó ningún overflow; es decir que el resultado de la suma fue menor que el máximo permitido.

Por otro lado, existe otra bandera denominada **Acarreo de Dígito DC** que expresa lo que sucede con los 4 Bits menos significativos; es decir, si los cuatro bits menos significativos sobrepasa al número 15 (2 elevado a 4, incluyendo el cero) expresado en forma decimal, entonces la bandera **DC** = 1, en el caso contrario será "0".

Finalmente la bandera de estado **Z** se activa cuando la operación aritmética da como resultado un "1"; de lo contrario se coloca en "0".

Las banderas en la resta

En la resta de dos números la bandera **CARRY** se coloca en "1" cuando el resultado de la operación sea un número positivo, o se pone en cero para el caso contrario. Esto tan sencillamente quiere decir por ejemplo que si tenemos $A=20$ y $B=10$ donde $X=A-B$; el resultado será $X=10$ (número positivo); para el caso contrario si tenemos $A=10$ y $B=20$ donde $X=A-B$ entonces $X= - 10$, obteniéndose un resultado negativo.

La bandera de acarreo de dígito **DC** se colocará en "1" cuando los cuatro bits menos significativos del registro **w** sea menor que los cuatro bits menos significativos del registro que se desea restar, en caso contrario se colocará un cero.

La bandera de estado **Z** solamente se activará cuando ambas cantidades sean iguales.

Las banderas en la rotación

El microcontrolador PIC16F84A tiene disponibles dos instrucciones de rotación las cuales rotan los dígitos de un registro a la derecha o a la izquierda.

Por ejemplo, para la rotación a la izquierda supongamos que nuestro registro **f** = 0000 1111 b y que la bandera **CARRY** tiene un 0; cuando se aplica el comando **RLF f,0** todos los números del registro **f** se desplazan hacia la izquierda. El valor lógico que se encuentra en la bandera **CARRY** es colocado en el **bit 0**, y el **bit 7** es colocado en la bandera **CARRY**. El resultado de nuestro ejemplo sería: **f** = 0001 1110 b y **CARRY** sería 0:

	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0	CARRY
	0	0	0	0	1	1	1	1	0
RLF f,d	0	0	0	1	1	1	1	0	0

Ahora para la rotación a la derecha, supongamos que nuestro registro **f** = 0000 1111 b y que la bandera **CARRY** tiene un 0; cuando se aplica el comando **RRF f,0** todos los números del registro **f** se desplazan hacia la derecha. El valor lógico que se encuentra en la bandera CARRY es colocado en el bit 7 y el bit. 0 es colocado en la bandera **CARRY**. El resultado de nuestro ejemplo seria: **f** = 0000 0111 b y **CARRY** seria 1:

	bit 7	bit 6	bit 5	bit 4	bit 3	bit 2	bit 1	bit 0	CARRY
	0	0	0	0	1	1	1	1	0
RRF f,d	0	0	0	0	0	1	1	1	1

Operaciones de comparación

Las operaciones de comparación utilizan la instrucción de resta. La resta no es mas que sumar al minuendo el complemento a 2 del sustraendo.

Igualdad

Supongamos que estamos intentando determinar si un número es igual a 2.

```
MOVLW .2
SUBWF N, W      ; W = N - 2
BTFSS STATUS, Z
GOTO NO_ES_IGUAL
GOTO ES_IGUAL
```

Al número a comprobar (N) se le resta la cantidad de comparación (2) que se ha guardado en W. El resultado vuelve a guardarse en W para salvaguardar la variable N. Finalmente se comprueba la bandera Zero del registro Status.

Mayor que y menor que

Supongamos que estamos intentando determinar si un número mayor o menor de 2.

```
MOVLW .2
SUBWF N, W      ; W = N - 2
BTFSS STATUS, C
GOTO MENOR
GOTO MAYOR_IGUAL
```

Aquí se comprueba la bandera C. Si Carry es 1 el resultado es positivo y si es 0 es negativo. Así, si $N < 2$ entonces el programa va a MENOR. Si N es igual o mayor a 2 se salta a MAYOR_IGUAL.

Así, el código anterior:

- $N < 2$ MENOR
- $N \geq 2$ MAYOR_IGUAL

Otra posibilidad es la siguiente:

```
MOVWF N, W
SUBLW .2      ; W = 2 - N
BTFSS STATUS, C
GOTO MAYOR
GOTO MENOR_IGUAL
```

Subrutinas

El movimiento del programa se basa en la llamada a etiquetas y a subrutinas. Esto nos da capacidad para decidir, temporizar o retardar, operar y/o ejecutar algoritmos. Por eso debemos tener en cuenta las técnicas más comunes en la programación de lenguaje ensamblador que nos permitirán hacer estas y muchas otras cosas.

La mayoría de los microcontroladores incluyen en su repertorio de instrucciones algunas que permiten saltar a una rutina y, cuando se completa su ejecución, retornar al programa principal.

El empleo de subrutinas aporta muchas ventajas entre las que se destacan las siguientes:

- Se pueden escribir como subrutinas secciones de código y ser empleadas varias veces en el mismo programa.
- Dan a los programas un carácter modular, es decir, se pueden codificar diferentes módulos para utilizarlos en cualquier programa.
- Se reduce notablemente el tiempo de programación y la detección de errores, utilizando repetidamente una subrutina.
- El código es más fácil de interpretar, dado que las instrucciones de las subrutinas no aparecen en el programa principal, sólo figuran las llamadas (**CALL**).

La instrucción CALL

La instrucción **CALL** (llamada a subrutina) consigue que la ejecución del programa continúe en la dirección donde se encuentra la subrutina a la que hace referencia. Es similar a **GOTO** pero coloca en la pila la dirección de la siguiente instrucción que se debe ejecutar después de terminar con la subrutina.

La subrutina finaliza con la instrucción **RETURN** (retorno de la subrutina) que retoma la dirección guardada en la pila y la coloca en el contador de programa **PC** continuando el flujo de control con la instrucción que sigue a **CALL**.

En la familia PIC de gama media la pila tiene ocho niveles de memoria del tipo LIFO (Last In, First Out, último en entrar, primero en salir). Si se produce la llamada a una subrutina durante la ejecución de otra subrutina, la dirección de retorno de esta segunda es colocada

en la cima de la pila sobre la dirección anterior. Esta segunda dirección es la primera en salir de la pila mediante la instrucción **RETURN** .

Con la pila de ocho niveles, una subrutina puede llamar a otra y ésta, a su vez, llamar a otra hasta un máximo de ocho.

Consulta a tablas

En muchas ocasiones es necesario efectuar una coincidencia entre alguna cantidad de valores conocidos y un número desconocido que se tiene como índice.

Por ejemplo, basados en el contenido de una posición de memoria RAM, que usaremos como índice, se puede obtener de una serie consecutiva de datos almacenados en la memoria de programa. A este conjunto de datos que queremos obtener a cambio de un valor del índice se les denomina tabla.

La técnica consiste en cargar el valor del índice en el acumulador, y después llamar a una subrutina que primero suma este valor al **PCL** , por lo cual obtendremos un desplazamiento de tantas líneas como indique el índice. Una vez nos hayamos desplazado hasta la línea deseada, esta indicará el fin de la subrutina, y devolverá en el acumulador el valor deseado, para ese valor del índice.

Veamos un ejemplo:

```
INDICE EQU      0Ch      ; posición de memoria RAM
.....
.....

        MOVF      INDICE,W      ;llevamos a W el número utilizado como índice
        CALL      TABLA        ; posición en donde se encuentra la serie de
                                ; datos. En esta línea se tiene en w el dato
                                ; leído de la tabla después del retorno de la
                                ; subrutina
.....
.....

TABLA
        ADDWF     PCL,f      ; se suma al PC W obteniendo como resultado un salto
                                ; indexado
        RETLW     30h        ; sí W sumado a PCL es 0 se retorna en esta posición,
                                ; W=30h
        RETLW     31h        ; sí W sumado a PCL es 1 se retorna en esta posición,
                                ; W=31h
        RETLW     32h        ; sí W sumado a PCL es 2 se retorna en esta posición,
                                ; W=32h
        RETLW     33h        ; sí W sumado a PCL es 3 se retorna en esta posición,
                                ; W=33h
        RETLW     34h        ; sí W sumado a PCL es 4 se retorna en esta posición,
                                ; W=34h
        RETLW     35h        ; sí W sumado a PCL es 5 se retorna en esta posición,
                                ; W=35h
```

Para terminar, después de observar el ejemplo anterior, debemos tener en cuenta que antes de llamar a la subrutina **TABLA**, se debe cargar en el registro de trabajo **w** el valor del índice y una vez se retorne de dicha subrutina, es en este mismo registro de trabajo en donde se obtiene el resultado de la consulta a la tabla (vemos que la sucesión de instrucciones **RETLW k** se encuentra en memoria de programa).

Conversión a ASCII

Códigos ASCII:

Dec	Hx	Char	Dec	Hx	Html	Chr	Dec	Hx	Html	Chr	Dec	Hx	Html	Chr
0	0	NUL (null)	32	20	 	Space	64	40	@	@	96	60	`	`
1	1	SOH (start of heading)	33	21	!	!	65	41	A	A	97	61	a	a
2	2	STX (start of text)	34	22	"	"	66	42	B	B	98	62	b	b
3	3	ETX (end of text)	35	23	#	#	67	43	C	C	99	63	c	c
4	4	EOT (end of transmission)	36	24	$	\$	68	44	D	D	100	64	d	d
5	5	ENQ (enquiry)	37	25	%	%	69	45	E	E	101	65	e	e
6	6	ACK (acknowledge)	38	26	&	&	70	46	F	F	102	66	f	f
7	7	BEL (bell)	39	27	'	'	71	47	G	G	103	67	g	g
8	8	BS (backspace)	40	28	(	(	72	48	H	H	104	68	h	h
9	9	TAB (horizontal tab)	41	29	)	)	73	49	I	I	105	69	i	i
10	A	LF (NL line feed, new line)	42	2A	*	*	74	4A	J	J	106	6A	j	j
11	B	VT (vertical tab)	43	2B	+	+	75	4B	K	K	107	6B	k	k
12	C	FF (NP form feed, new page)	44	2C	,	,	76	4C	L	L	108	6C	l	l
13	D	CR (carriage return)	45	2D	-	-	77	4D	M	M	109	6D	m	m
14	E	SO (shift out)	46	2E	.	.	78	4E	N	N	110	6E	n	n
15	F	SI (shift in)	47	2F	/	/	79	4F	O	O	111	6F	o	o
16	10	DLE (data link escape)	48	30	0	0	80	50	P	P	112	70	p	p
17	11	DC1 (device control 1)	49	31	1	1	81	51	Q	Q	113	71	q	q
18	12	DC2 (device control 2)	50	32	2	2	82	52	R	R	114	72	r	r
19	13	DC3 (device control 3)	51	33	3	3	83	53	S	S	115	73	s	s
20	14	DC4 (device control 4)	52	34	4	4	84	54	T	T	116	74	t	t
21	15	NAK (negative acknowledge)	53	35	5	5	85	55	U	U	117	75	u	u
22	16	SYN (synchronous idle)	54	36	6	6	86	56	V	V	118	76	v	v
23	17	ETB (end of trans. block)	55	37	7	7	87	57	W	W	119	77	w	w
24	18	CAN (cancel)	56	38	8	8	88	58	X	X	120	78	x	x
25	19	EM (end of medium)	57	39	9	9	89	59	Y	Y	121	79	y	y
26	1A	SUB (substitute)	58	3A	:	:	90	5A	Z	Z	122	7A	z	z
27	1B	ESC (escape)	59	3B	;	;	91	5B	[	[	123	7B	{	{
28	1C	FS (file separator)	60	3C	<	<	92	5C	\	\	124	7C	|	
29	1D	GS (group separator)	61	3D	=	=	93	5D	]	]	125	7D	}	}
30	1E	RS (record separator)	62	3E	>	>	94	5E	^	^	126	7E	~	~
31	1F	US (unit separator)	63	3F	?	?	95	5F	_	_	127	7F		DEL

Pantalla de código ASCII extendido en el PC IBM original:

```

0  0 1 2 3 4 5 6 7 8 9 A B C D E F
1  ▶ ◀ ⚡ ⚡ ⚡ ⚡ ⚡ ⚡ ⚡ ⚡ ⚡ ⚡ ⚡ ⚡ ⚡ ⚡
2  ! " # $ % & ' ( ) * + , - . /
3  0 1 2 3 4 5 6 7 8 9 : ; < = > ?
4  @ A B C D E F G H I J K L M N O
5  P Q R S T U V W X Y Z [ \ ] ^ _
6  ` a b c d e f g h i j k l m n o
7  p q r s t u v w x y z { | } ~
8  ª « ¬ ® ¯ ° ± ² ³ ´ µ ¶ · ¸ ¹ º
9  » ¼ ½ ¾ ¿
A  Á Â Ã Ä Å Æ Ç È É Ê Ë Ì Í Î Ï
B  Ñ Ò Ó Ô Õ Ö × Ø Ù Ú Û Ü Ý Þ ß
C  à á â ã ä å æ ç è é ê ë ì í î ï
D  ð ñ ò ó ô õ ö ÷ ø ù ú û ü ý þ ÿ
E  Æ Ç È É Ê Ë Ì Í Î Ï
F  Ñ Ò Ó Ô Õ Ö × Ø Ù Ú Û Ü Ý Þ ß

```

Pantalla de código ASCII extendido en Windows

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	■	■	■	■	■	■	■					■	■		■	■
1	■	■	■	■	■	■	■	■	■	■		■	■	■	■	■
2		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	■
8	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■	■
9	■	'	'	■	■	■	■	■	■	■	■	■	■	■	■	■
A		;	;	£	¥	¥	¥	¥	¥	¥	¥	¥	¥	¥	¥	¥
B	°	±	±	±	±	±	±	±	±	±	±	±	±	±	±	±
C	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
D	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
E	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
F	ø	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	

El conjunto de caracteres ASCII (American Standard Code for Information Interchange) es el código de representación en hexadecimal del alfabeto, los números del 0 al 9, los principales símbolos de puntuación y algunos caracteres de control.

Como vemos en la tabla ASCII, podemos dividir cada carácter representado en hexadecimal como una parte alta de 3 bits (Most Significant Character = números del 0 al 7) y una parte baja de 4 bits (Least Significant Character = números del 0 al F). En total, la representación la hacemos con 7 bits.

Los códigos ASCII menores de 32 (decimal, de 00h a 1Fh) son los llamados caracteres de control. Se utilizan como comandos en los dispositivos serie y paralelo (terminales, impresoras, etc) efectuando operaciones como: avance de papel, retorno de carro, fin de transmisión, fin de archivo, etc. En condiciones normales, por ejemplo en un editor de texto ASCII, son caracteres que no pueden representarse gráficamente.

De los problemas más frecuentes en la programación, está el de convertir un número hexadecimal representado en 8 bits a dos caracteres ASCII los cuales sean la representación de dicho número para permitir su visualización en pantallas LCD, monitores, impresoras, etc.

Por, ejemplo, queremos representar el número hexadecimal 70 h, que en binario es 01110000 b y en ASCII necesita dos caracteres, "7" y "0"

Gráficamente:

7	0	en hexadecimal (8 bits)
"7"	"0"	en ASCII (16 bits)
37h	30h	ASCII en hexadecimal (16 bits)

Lo transportamos a un programa:

```

numHEX EQU    0Ch    ; posición donde se almacena el número a convertir
asciiH EQU    0Dh    ; posición donde se almacena el resultado parte alta
asciiL EQU    0Eh    ; posición donde se almacena el resultado parte baja

```

```

.....
.....

```

```

                                ; dato para enmascarar parte alta
                                ; se enmascara la parte alta del
número                          MOV LW 0Fh
                                ANDWF numHEX,0
                                ; hexa y pasa a W
                                ; convierte el número en ASCII
                                ; número salvado en la variable de
salida                          IORLW 30h
                                MOVWF asciiL
                                ; dato para enmascarar parte baja
                                ; se enmascara la parte baja del
número                          MOV LW F0h
                                ANDWF numHEX,1
                                ; hexadecimal y queda allí
                                ; se invierten parte alta y baja
                                ; convierte el número en ASCII
                                ; el número queda salvado en la
variable de                     SWAPF numHEX,0
                                IORLW 30h
                                MOVWF asciiL
                                ; salida

```

```

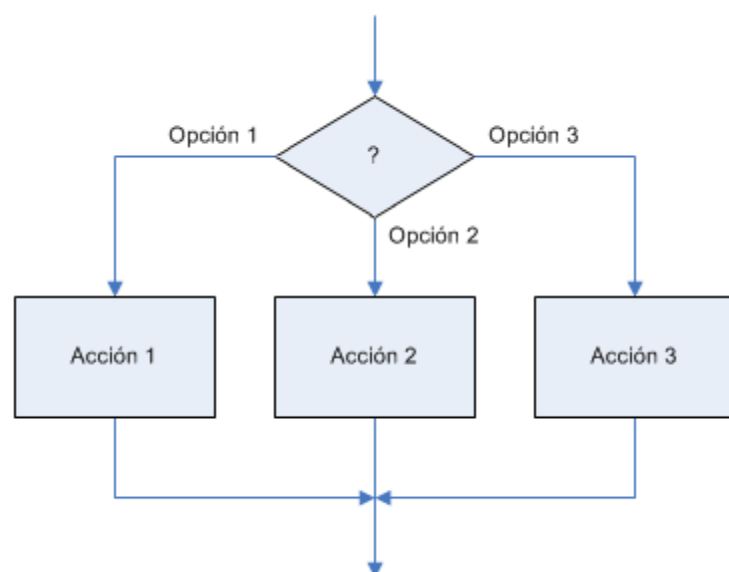
.....
.....

```

El ejemplo anterior funciona de forma correcta siempre y cuando los nibbles del número hexadecimal a convertir estén en el rango de 0 a 9. Habrá que realizar un tratamiento adicional a éstos si se encuentran en el rango de A h a F h.

Ramificación múltiple

Cuando se tiene que solucionar un diagrama de flujo como el de la siguiente figura, en el cual tenemos tres posibles respuestas a una pregunta, se plantean las soluciones aquí presentadas.



Existen varias formas de resolver en un programa este problema:

Solución 1

Determinamos para la opción 1, la opción 2 y la opción 3 un valor consecutivo como:

```
OPCION1 EQU    0
OPCION2 EQU    1
OPCION3 EQU    2
```

Uno de estos posibles valores lo llevamos a **w** y en una parte del programa los tratamos de la siguiente manera:

```
                ADDWF    PCL,1
                GOTO     ACCION1
                GOTO     ACCION2
                GOTO     ACCION3

ACCION1:        .....      ;instrucciones correspondientes a la Acción 1
                .....
                GOTO     ENCUESTRO

ACCION2:        .....      ;instrucciones correspondientes a la Acción 2
                .....
                GOTO     ENCUESTRO

ACCION3:        .....      ;instrucciones correspondientes a la Acción 3
                .....

ENCUESTRO      .....      ; sitio de encuentro de los distintos caminos
                  ; después de una de las acciones
                .....      ;continuación del programa
```

Solución 2

Otra forma posible es comparando uno por uno los valores de las diferentes opciones almacenadas en memoria RAM en una variable llamada OPCION

```
                MOVLW    OPCION1
                XORWF    OPCION,0      ; verificación de OPCION respecto a W
                BTFSC    STATUS,Z      ; verificando la bandera Z
                GOTO     ACCION1
                MOVLW    OPCION2
                XORWF    OPCION,0      ; verificación de OPCION respecto a W
                BTFSC    STATUS,Z      ; verificando la bandera Z
                GOTO     ACCION2
                MOVLW    OPCION3
                XORWF    OPCION,0      ; verificación de OPCION respecto a W
                BTFSC    STATUS,Z      ; verificando la bandera Z
                GOTO     ACCION3

ACCION1        .....      ; instrucciones de la Acción 1
                .....
                .....
                GOTO     ENCUESTRO

ACCION2        .....      ; instrucciones de la Acción 2
                .....
```

```

.....
GOTO  ENCuentro

ACCION3      .....      ; instrucciones de la Acción 3
              .....
              .....

ENCuentro    .....      ; sitio de encuentro
              .....      ; continuación del programa
              .....

```

Aunque este último método es más largo que el anterior, es válido cuando los valores de las diferentes opciones no son consecutivos entre si.

Temporización

Hay veces en las que se necesita introducir ciertos retardos de tiempo. Los retardos de tiempo se pueden obtener mediante hardware o por medio de ciclos repetitivos basados en software.

Los retardos de tiempo basados en software se realizan mediante un bucle e incrementando o disminuyendo un contador que cuando pase por cero hará que salgamos de la condición.

Como ya sabemos, un ciclo máquina es el tiempo utilizado por el microcontrolador para realizar sus operaciones internas y equivale a 4 ciclos de reloj u oscilador.

Por tanto:

$$T_{\text{ciclo máq.}} = 4 * T_{\text{osc}}$$

$$T_{\text{ciclo máq.}} = 4 / f_{\text{osc}}$$

Como cada instrucción necesita 4 ciclos de reloj para que se ejecute, si usamos un cristal de 4 MHz cada instrucción ocupará 1 microsegundo, a no ser que el contador del programa se modifique.

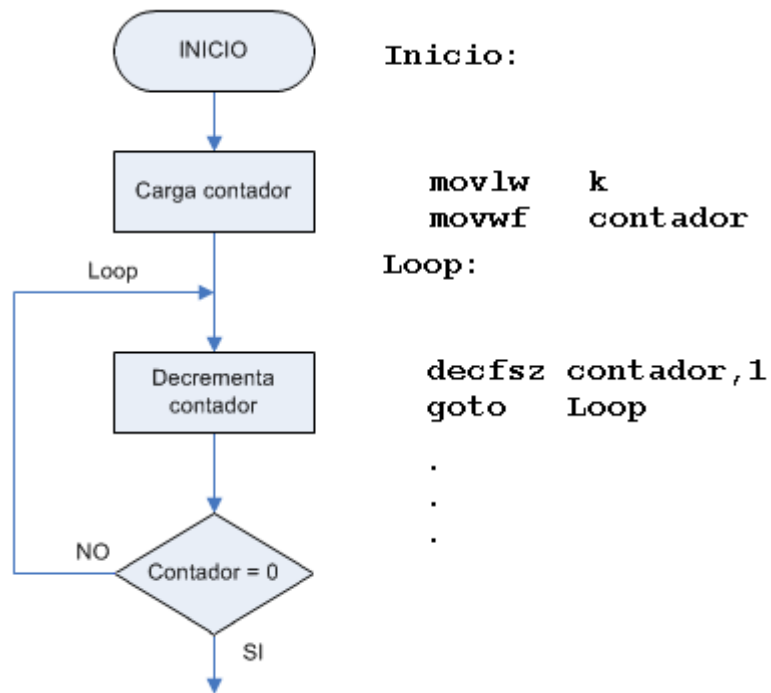
El número de ciclos máquina utilizados por una instrucción para ser ejecutada depende de la misma. Las instrucciones que modifican el contador de programa necesitan dos ciclos máquina, mientras que todas las demás necesitan solo uno. De esta manera las instrucciones de salto necesitan 2 ciclos máquina para ejecutarse.

La precisión de los retardos generados por software depende en esencia del tipo de oscilador que se utilice como base de tiempo en el microcontrolador (la mayor precisión se obtiene de los cristales de cuarzo).

La velocidad a la que se ejecuta el código (instrucciones) depende de la velocidad del oscilador y del número de ciclos máquina ejecutados. Las instrucciones necesitan 1 ó 2 ciclos de máquina para ser ejecutadas.

El hecho de generar ciclos repetitivos por medio del programa y calcular el tiempo total de ejecución nos puede ayudar a generar tiempos precisos.

Un ejemplo de ciclo repetitivo lo tenemos a continuación, en la siguiente figura:



Este algoritmo consume ciclos de la siguiente manera:

OPERACIÓN	CICLOS
la carga de k en W	1 c
la carga de W en el contador	1 c
el decremento del contador mientras no llegue a cero	k - 1
el decremento del contador cuando llegue a cero	2 c
el salto a Loop	2 * (k - 1)
Total:	(3 * k) + 1

Por cada instrucción agregada debe incluirse en la cuenta total el número de ciclos correspondiente a dicha instrucción.

Trabajando a 4 Mhz y asumiendo que k se remplaza por el valor 15 en decimal en el ejemplo tendríamos un tiempo igual a:

$$\text{Número de ciclos} = (3 \cdot 15) + 1 = 46 \text{ ciclos máquina}$$

$$T_{\text{ciclo máq.}} = 4 / 4 \text{ Mhz} = 1 \mu\text{s}, \text{ el tiempo total entonces será de } 46 \mu\text{s}.$$

Veamos como ejemplo las rutinas **MSEC1** y **MIC4**. Con un cristal de 4 MHz, MIC4 tarda en ejecutarse 4 microsegundos y haciendo uso de esto, MSEC1 proporciona un retardo de 1 milisegundo al ejecutar 249 veces MIC4:

```

MSEC1  MOVLW    0xF9          ; carga F9 en el acumulador 249 en decimal
        NOP                ; por la llamada a la subrutina CALL MSEC1
MIC4    ADDLW    0xFF          ; subtrae 1 de W
        BTFSS    STATUS,Z      ; salta cuando llega a cero
        GOTO     MIC4          ; si no llega a cero vuelve a restar
        RETURN

```

Un milisegundo son 1000 microsegundos, de manera que necesitamos ocupar 1000 ciclos de reloj en la subrutina, que hemos llamado **MSEC1**.

El bucle **MIC4 - GOTO MIC4** necesita 4 microsegundos para ejecutarse:

- **ADDLW** toma 1 microsegundo
- **BTFSS** toma otro microsegundo
- **GOTO** necesita 2 microsegundos

Para restar 1 al acumulador se utiliza **ADDLW 0xFF**, en lugar de **SUBLW 0x1** porque esta instrucción no resta el literal a **w**, sino al revés, al literal le resta **w**. Por lo tanto para restar un literal de **w** debemos sumar el complemento a 2 del literal con w, en nuestro caso el literal es 1 (0000 0001 b) y el complemento a 2 de 1 es FF h:

```

0000 0001
1111 1110
    +1
-----
1111 1111 (FF h.)

```

Después de restar, la subrutina MIC4, comprueba la bandera **Z** en el registro **STATUS**, que será puesto a uno cuando la resta sea 0. La comprobación del bit tarda un microsegundo a menos que se realice el salto, en cuyo caso se efectúa en 2 microsegundos.

Ciclos de instrucción (c) de la subrutina:

ETIQUETA	INSTRUCCIÓN	CICLOS	
MSEC1	MOVLW 0xF9	1 c	
	NOP	1 c	
MIC4	ADDLW 0xFF	1c	TOTAL MIC4 => (249 * 4c) + 1c = 996 c
	BTFSS STATUS,Z	1c, 2c al saltar	
	GOTO MIC4	2 c	

	RETURN	2 c
		Total, 1000 c

Como puede observarse después de ejecutar **CALL MSEC1** transcurrirán 1000 ciclos de reloj, esto es 1 milisegundo antes de pasar a la siguiente instrucción.

La subrutina no utiliza ningún registro aparte de **w**. Para periodos de tiempo más largos deberán utilizarse registros.

La siguiente rutina es llamada con el número de milisegundos que deberán transcurrir dentro del acumulador según el valor de la variable **CNTMSEC**. Hace uso de la rutina **MIC4**. Se pueden realizar retardos de hasta un cuarto de segundo(1 - 255 msec):

```

NMSEC  MOVWF  CNTMSEC      ; mueve W al registro msec
MSLOOP MOVLW  0xF8         ; cuenta 8 microsegundos por encima
      CALL  MIC4           ; 248 * 4 + 2 = 994
      NOP                ; realiza el resto del bucle
      NOP                ; añade 6 microsegundos
      DECFSZ CNTMSEC, f    ; decrementa el contador
                        ; salta cuando llega a cero
      GOTO  MSLOOP        ; vuelve a realizar el bucle
      RETURN

```

Instrucciones y puertos

Conviene recordar que el PIC16F84A tiene 13 patillas que pueden ser configuradas individualmente como entrada o como salida. Están divididos en dos puertos de 8 patillas y otro de 5, puerto B y puerto A, respectivamente. La dirección de cada bit está determinada por los bits de los registros [TRISA](#) y [TRISB](#) del banco de memoria 1. Un cero en un bit significa que es una salida, mientras que un uno significa que queda configurado como una entrada.

Ejemplo de cómo configurar el puerto B alternando entradas y salidas:

```

BSF    STATUS,RP0        ; Activa el banco de memoria 1.
MOVLW  0xAA              ; coloca en el acumulador el valor '10101010'
MOVWF  TRISB              ; W es copiado en el registro TRISB
BCF    STATUS,RP0        ; Activa el banco de memoria 0.

```

No se recomienda utilizar la instrucción TRIS, ver [Instrucciones OPTION y TRIS](#).

Recuerdese que algunas patillas de los puertos están relacionados con otras funciones del microcontrolador. Los 4 bits más altos del puerto B pueden ser utilizados como interrupciones cuando son programados como entradas. El bit 0 del puerto B también puede ser usado como fuente de interrupción externa. El bit más alto del puerto A puede utilizarse también como entrada externa de reloj para el contador-temporizador.

Ver [Puertos de E/S](#) de "El PIC16F84A" para observar la constitución interna de los puertos y su funcionamiento (corrientes de salida que proporcionan, etc).

Escritura en los puertos

Antes de nada, ya sabemos que podemos ajustar independientemente cada línea de un puerto para que sea entrada o salida, mediante la instrucción **TRIS** . Así, para tener más control sobre lo que estamos haciendo, podemos cargar en este registro el valor en binario.

Ejemplo, en lugar de:

```
MOVLW    30h
MOVWF    TRISB
```

Ponemos:

```
MOVLW    B'00110000'
MOVWF    TRISB
```

Para escribir en los puertos, podemos mover directamente el valor hexadecimal desde el acumulador al puerto entero. Esto se utiliza en aquellos casos en que usemos un puerto entero como un bus de datos, como puede ser en un display. Pero lo más normal es controlar cada patilla activando o desactivando independientemente los bits del registro **PORTA** o **PORTB** , a través de la instrucción BSF (activa) y BCF (desactiva).

Ejemplo, mover un literal al puerto:

```
MOVLW    0F
MOVWF    TRISB
```

Activando bits del puerto:

```
BSF      TRISB, 0
BSF      TRISB, 1
BSF      TRISB, 2
BSF      TRISB, 3
```

Cuestiones a tener en cuenta

Los puertos que contienen entradas y salidas necesitan una atención especial al escribir el programa. Instrucciones como **BSF** y **BCF** comienzan leyendo el valor del puerto y cargándolo en **w**; allí ejecutan la puesta a 1 ó a 0 del bit seleccionado y, luego, depositan el registro **w** en el puerto. También hay que tener en cuenta las modificaciones que se produzcan en las patillas que son entrada y pasan a salida, pues pueden estar presentes datos antiguos en el registro de salida del puerto al ser memorizados.

Hay que prestar mucha atención a las operaciones en las que, tras una lectura de un puerto, sigue una escritura de la misma. Se debe dejar pasar un tiempo determinado para que se establezca el voltaje de las salidas. Insertando entre la lectura y la escritura una instrucción **NOP** o cualquier otra que no implique a los puertos, seremos capaces de eliminar estos errores potenciales.

Lectura de los bits del puerto

Pulsadores y anti-rebotes

Al igual que hemos escrito en los puertos a través de dos métodos distintos, igualmente podemos leer de ellos usando ambas metodologías.

El primero resulta bastante obvio, basta con realizar el proceso inverso: movemos el valor del puerto a **w** y de ahí a donde queramos hacer uso de ese valor.

El segundo se basa en las instrucciones que preguntan sobre el estado de un bit, esto es, **BTFS** y **BTFS**. Y dependiendo del bit, una realiza un salto si está a 0 y la otra si está a 1. En el siguiente ejemplo lo vemos más claro.

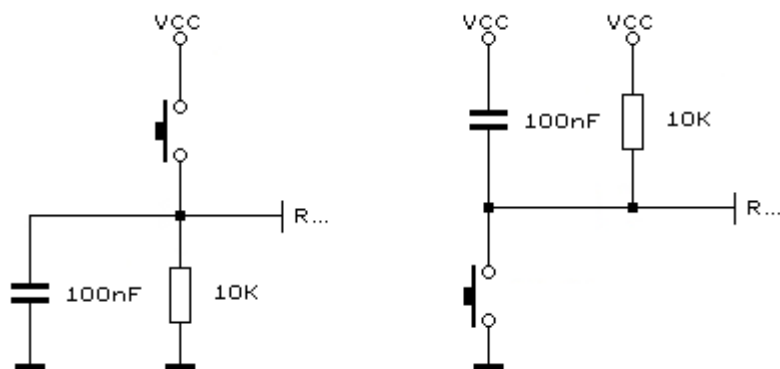
```
PRUEBA BTFS    PORTA, 0      ; comprueba el estado del bit 0 del puerto A
                                GOTO  PRUEBA ; si no cambia, vuelve a comprobarlo
                                GOTO  OTRO   ; si cambia, sale del bucle y va a
otro lado
OTRO            .....      ; otras instrucciones
```

Esto está bien en el caso de que lo apliquemos a entradas basadas en circuitería lógica, o que cambian de estado una vez cada mucho tiempo. Si quisiésemos aplicarlo, por ejemplo, a una entrada a la que tenemos conectado un pulsador, hemos de usar un circuito y un algoritmo anti-rebotes.

Los rebotes son de sobra conocidos por todos aquellos que están iniciados en la electrónica digital. Son producidos por los elementos electromecánicos conectados a un circuito lógico, los cuales causan que este funcione mal al ser pulsados, ya que estos provocan un tren de pulsos debido a los rebotes que provocan las partes móviles al tomar contacto con las fijas.

Hardware para evitar rebotes

Existen varias soluciones, según utilicemos entradas de nivel alto o de nivel bajo. Pero todas tienen en común una resistencia de Pull-Up (o de Pull-Down) y un condensador de unos 100nF que absorberá cualquier interferencia y/o rebote en la línea de entrada. En la figura siguiente se muestran distintas configuraciones para evitar los rebotes, una con el condensador a masa, y la otra con el condensador a Vcc. En ciertas ocasiones es normal situar una resistencia de unos 270 ohmios a 1K, entre el circuito y la patilla del PIC.



Si bien usando resistencias de pull-up, podemos salvar estas situaciones, hemos de tener presente que todas las líneas del puerto B disponen de estas resistencias, por lo cual no será necesario añadirlas, ya que están implementadas. Para activarlas basta con poner a cero el bit **RBPU** del registro **OPTION**.

Software para evitar rebotes

Más que para evitar los rebotes, esta aplicación se usa para no pasar de instrucción sin antes haber soltado el pulsador, ya que no somos tan rápidos como para que la entrada esté a nivel bajo antes de una supuesta comprobación del mismo bit. La solución está en quedarnos bloqueados en otro bucle hasta que hayamos liberado el pulsador.

Ejemplo:

```
BUCLE          BTFSS  PORTA,1 ; bucle infinito
                GOTO   BUCLE   ; mientras no activemos el pulsador
ESPERAR BTFSC   PORTA,1      ; una vez que lo hayamos pulsado y salido
                GOTO   ESPERAR ; entramos en otro bucle a esperar a soltarlo
                BSF     PORTA,0 ; una vez liberado, por ejemplo encendemos un
                                ; led
```

Otras operaciones en los puertos

Recuerda que los puertos están controlados por los registros **PORTA** y **PORTB**, y que estos registros se comportan como un registro cualquiera. Con esto podemos operar con ellos, activan banderas, etc.

Por esto, otro tipo de operaciones diferentes a las ya vistas, serán válidas en los casos en los que operemos con un puerto entero bajo un mismo bus de datos, por ejemplo. Podemos usar instrucciones de suma y resta, álgebra de Boole, rotación, incrementación y decrementación, etc.
